
DSC 40B - Midterm 02 Review

Problem 1.

The goal of contact tracing is to determine how the spread of a virus occurs. Which type of graph would be best for modelling the spread of a virus?

- ☐ Directed graph
- ☐ Undirected graph

Problem 2.

A directed graph has 7 nodes. What is the maximum number of edges it can have?

Problem 3.

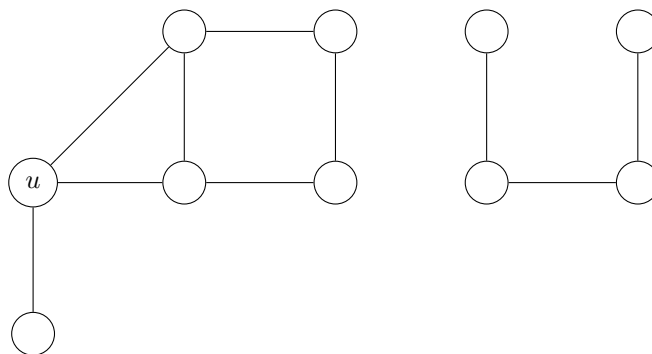
An undirected graph has 12 nodes. What is the maximum number of connected components it can have?

Problem 4.

A directed graph has 5 nodes. What is the largest degree that a node in the graph can possibly have?

Problem 5.

How many nodes are reachable from node u in the graph?



Problem 6.

Assume that a hash table is implemented using chaining to resolve collisions. The hash table stores n numbers. Suppose a membership query is made for 42.

True or False: more than one bin in the hash table may be checked with a linear search during the query.

☐ True

☐ False

Problem 7.

Suppose a hash table is constructed so that collisions are resolved through chaining. The hash table has a number of bins k that is $\Theta(n)$, where n is the number of elements being stored in the hash table; that is, the hash table grows as more elements are inserted.

Suppose the hash function uses only the first $k/2$ bins of the hash table, but appears to hash items into these bins evenly. What is the expected time complexity of a membership query on this hash table? State your answer in asymptotic notation in terms of the number of elements in the hash table, n .

Problem 8.

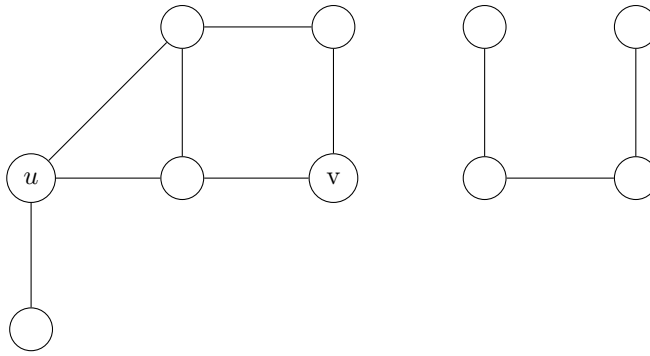
Both Full BFS and Full DFS can be used to count the number of connected components in an undirected graph.

☐ True

☐ False

Problem 9.

How many paths are there from node u to node v in the graph below?



- ☐ Infinitely many
- ☐ 4
- ☐ 3
- ☐ 5

Problem 10.

In an unweighted graph, there is at most one shortest path between any pair of given nodes.

- ☐ True
- ☐ False

Problem 11.

An undirected graph has 5 nodes. What is the smallest number of connected components it can have?

Problem 12.

In a full BFS of a graph $G=(V, E)$, the number of times that something is popped from the queue is $2V$ if the graph is undirected and V if the graph is directed.

- ☐ True
☐ False

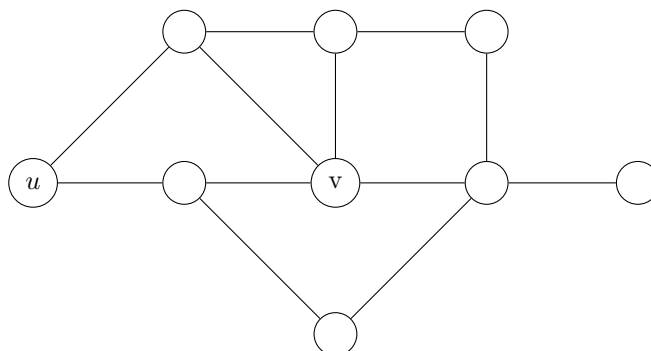
Problem 13.

In BFS it is possible for the queue to simultaneously contain a node whose distance from the source is 3 and node whose distance from the source is 5.

- ☐ True
☐ False

Problem 14.

Suppose a BFS is run on the graph below with u as the source.



Of course, u is the first node to be popped of the queue. Suppose that node v is the k th node popped from the queue.

a) What is the smallest that k can possibly be?

b) What is the largest that k can possibly be?

Problem 15.

Consider the modified BFS given below:

```
def bfs(graph, source, status=None):
    if status is None:
        status = {node: 'undiscovered' for node in graph.nodes}

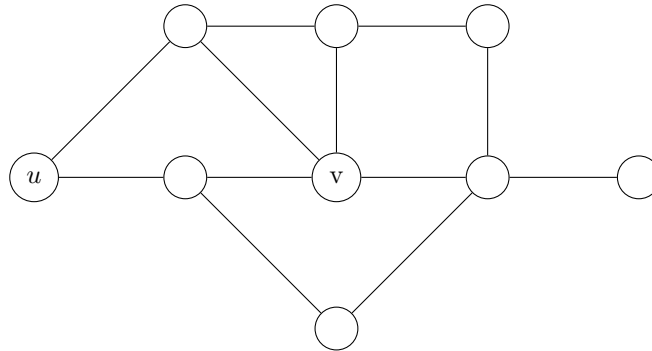
    status[source] = 'pending'
    pending = deque([source])

    # while there are still pending nodes
    while pending:
        u = pending.popleft()
        for v in graph.neighbors(u):
            # explore edge (u,v)
            if status[v] == 'undiscovered':
                print("Hey")
                status[v] = 'pending'
                # append to right
                pending.append(v)
        status[u] = 'visited'
```

Suppose this code is run on a connected undirected graph with 12 nodes. Exactly how many times will 'Hey' be printed?

Problem 16.

Suppose a DFS is run on the graph below with u as the source.



Node u will be the first node marked pending. Suppose that node v is the k th node marked pending.

a) What is the smallest that k can possibly be?

b) What is the largest that k can possibly be?

Problem 17.

If DFS is called on a complete graph, the time complexity is $\theta(V^2)$

- ☐ True
☐ False

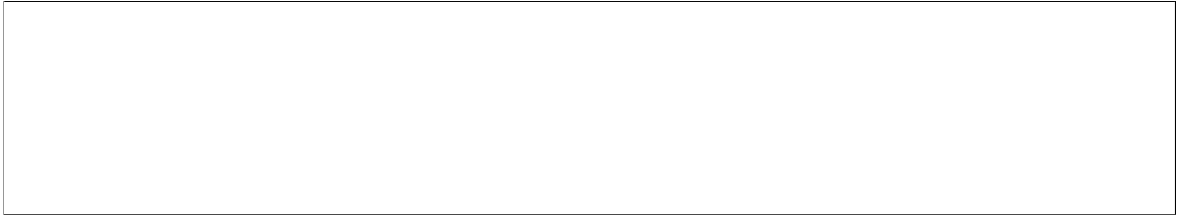
Problem 18.

What is the result of updating the edge (u,v) when the $est[u]$, $est[v]$ and $weight(u,v)$ are given as follows?

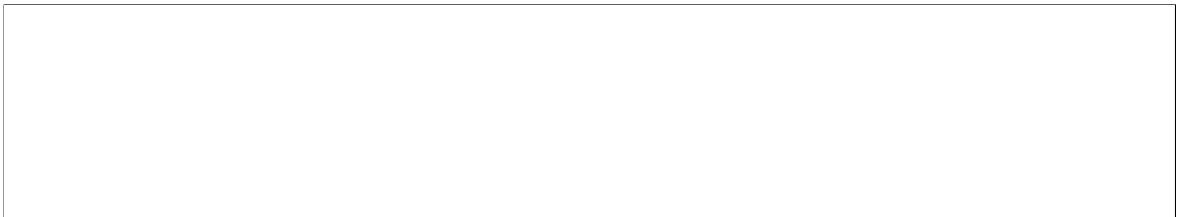
a) $est[u] = 7$, $est[v] = 11$, $weight(u,v) = 3$

Figure 1: Bellman Ford update subroutine

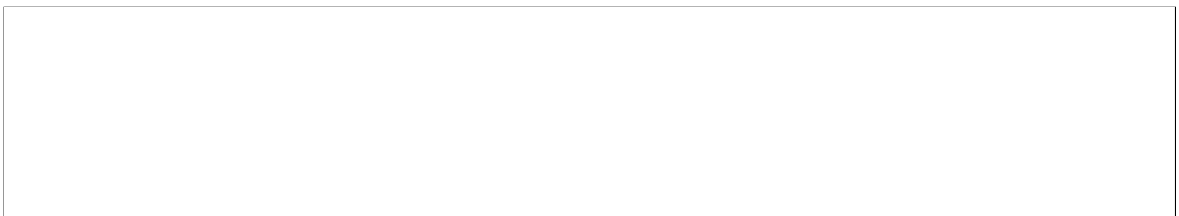
```
def update(u, v, weights, est, predecessor):  
    if est[v] > est[u] + weights(u,v):  
        est[v]=est[u]+weights(u,v)  
        predecessor[v]=u  
        return True  
    else:  
        return False
```



b) $\text{est}[u] = 15$, $\text{est}[v] = 12$, $\text{weight}(u,v) = -3$



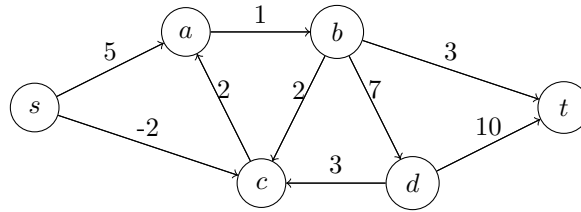
c) $\text{est}[u] = 12$, $\text{est}[v] = 14$, $\text{weight}(u,v) = 3$



Problem 19.

Run Bellman-Ford on the following graph using node s as the source. Below each node u, write the shortest path length from s to u. Mark the predecessor of u by highlighting it or making a bold arrow.

```
def bellman_ford(graph, weights, source):  
    est={node:float('inf') for node in graph.nodes}  
    est[source]=0  
    predecessor={node: None for node in graph.nodes}  
    for i in range(len(graph.nodes)-1):  
        for(u, v) in graph.edges:  
            update(u, v, weights, est, predecessor)  
    return est, predecessor
```



Problem 20.

State TRUE or FALSE for the following statements:

- a) If (s, v_1, v_2, v_3, v_4) is a shortest path from s to v_4 in a weighted graph, then (s, v_1, v_2, v_3) is a shortest path from s to v_3

- b) Let P be a shortest path from some vertex s to some other vertex t in a directed graph. If the weight of each edge in the graph is increased by one, P will still be a shortest path from s to t .

- c) Suppose the update function is modified such the $est[v]$ is updated when $est[v] \geq est[u] + weight(u,v)$ instead of strictly greater than. The est values of all nodes at the end of the algorithm would still give the shortest distance from the source.

- d)** Suppose the update function is modified such the $\text{est}[v]$ is updated when $\text{est}[v] \geq \text{est}[u] + \text{weight}(u,v)$ instead of strictly greater than. We can still find the shortest path from the source to any node using the predecessors using the new algorithm.